Duel Game

Documentație tehnică – TypeScript / Node.js

Balan Radu

1 Rulare program

După instalarea dependențelor, programul se poate rula din terminal.

```
npm install
```

Comenzi disponibile:

```
npm start
```

Rulează un singur duel, cu abilități fixe.

```
npm start -- --per-round
```

Rulează un singur duel, dar abilitatea se poate schimba la fiecare rundă.

```
npm start -- --simulate
```

Rulează 1000 de dueluri și afișează statistici.

```
npm start -- --simulate --per-round
```

Rulează 1000 de dueluri folosind modul cu abilitate per-rundă.

2 Cerințe implementate

2.1 Character Setup

Fiecare personaj este creat cu:

- 100 HP;
- attack random între 15 și 20;
- defense random între 10 și 15;
- o abilitate specială aleasă random.

Valorile importante sunt ținute în constante, nu scrise direct în cod ca *magic numbers*. Astfel, dacă regulile jocului se schimbă, modificările se fac într-un singur loc.

2.2 Special Abilities

Fiecare abilitate are 25% șansă de activare în momentul relevant.

Abilitate	**Moment**	**Efect**
Power Strike	Atac	Crește puterea atacului cu 50%.
Damage Reduction	Apărare	Reduce damage-ul primit la jumătate.
Second Wind	După damage	Dacă HP-ul scade sub pragul stabilit, personajul se vindecă cu 5 HP.

2.3 Combat Rules

Regulile de luptă sunt:

- se alege random cine atacă primul;
- atacatorul produce damage în funcție de attack;
- apărătorul reduce damage-ul prin defense;
- rolurile se inversează după fiecare atac;
- lupta se termină când unul dintre personaje ajunge la 0 HP;
- dacă un personaj moare după primul atac al rundei, nu mai contraatacă.

Formula de bază pentru damage este:

$$damage = max(1, attack - defense)$$

Am ales varianta cu minim 1 damage pentru ca lupta să avanseze mereu și să nu existe situații în care două personaje nu își pot scădea viața deloc.

3 Extensii opționale implementate

3.1 Simulare 1000 de jocuri

Flag-ul `-simulate` rulează 1000 de dueluri și afișează rezultatele:

- victorii pentru C1;
- victorii pentru C2;
- remize, dacă apar;
- procentajele de victorie.

În modul de simulare nu se afișează toate rundele, deoarece ar produce prea mult text inutil.

3.2 Abilitate fixă sau per-rundă

Programul acceptă flag-ul `-per-round`. Fără acest flag, fiecare personaj primește o abilitate la început și o păstrează până la final.

Cu `-per-round`, abilitatea este aleasă din nou în timpul luptei, ceea ce face duelul mai imprevizibil.

3.3 Adăugarea unei abilități noi

Codul este făcut astfel încât o abilitate nouă să poată fi adăugată fără să fie rescrisă logica principală de luptă.

Ideea este că fiecare abilitate își definește singură comportamentul prin hook-uri precum `onAttack`, `onDefend` sau `onAfterTakeDamage`. Funcția de combat doar verifică dacă aceste hook-uri există și le apelează la momentul potrivit.

Astfel, pentru o abilitate nouă se adaugă o nouă definiție în lista de abilități, fără să fie nevoie să se modifice partea principală a luptei.

4 Decizii de design

4.1 Abilități ca obiecte, nu switch mare

În loc să folosesc un `switch` mare pentru fiecare abilitate, fiecare abilitate este reprezentată ca un obiect cu funcții proprii.

De exemplu, o abilitate poate avea:

- `onAttack`;
- `onDefend`;
- `onAfterTakeDamage`.

Acest lucru face codul mai ușor de extins. Dacă adaug o abilitate nouă, nu trebuie să modific funcția principală care procesează atacul.

4.2 Constante pentru valorile jocului

Valorile precum HP-ul inițial, șansa de activare, limitele pentru attack și defense sunt definite ca valori constante.

Exemplu:

```
const STARTING_HEALTH = 100;
const ABILITY_CHANCE = 0.25;
const ATTACK_MIN = 15;
const ATTACK_MAX = 20;
```

Astfel, codul este mai clar și mai ușor de modificat.

4.3 Clamping pentru HP

După fiecare atac, HP-ul este limitat pentru a nu deveni negativ. De asemenea, după vindecare, HP-ul nu trebuie să depășească valoarea maximă.

Exemplu:

```
health = Math.max(0, health);
health = Math.min(MAX_HEALTH, health);
```

4.4 Limită de runde

Chiar dacă duelul ar trebui să se termine natural, există și o limită maximă de runde. Aceasta este o măsură de siguranță, utilă mai ales dacă în viitor se adaugă abilități mai defensive sau de vindecare.

5 Structura codului

Codul este împărțit în mai multe zone simple:

Nr.	Secțiune	Rol
1	Tipuri	Definește tipurile folosite: personaje, abilități, rezultate.
2	Constante	Conține valorile principale ale jocului.
3	Funcții helper	Funcții pentru random, șanse și limitarea valorilor.
4	Abilități	Definește cele trei abilități speciale.
5	Creare personaje	Inițializează C1 și C2 cu valori random.
6	Procesare atac	Aplică damage, defense și abilități.
7	Rulare duel	Alternează atacatorul și apărătorul până la final.
8	Simulare	Rulează 1000 de jocuri și calculează statistici.
9	Entry point	Citește argumentele din linia de comandă.

6 Exemplu de output

Un duel poate afișa informații de forma:

```
C1: HP = 100, attack = 18, defense = 12, ability = Power Strike
C2: HP = 100, attack = 16, defense = 14, ability = Damage Reduction

Round 1
C1 attacks C2
C1 activates Power Strike
C2 has 90 HP

Round 2
C2 attacks C1
```

```
No ability activated
C1 has 97 HP

C1 won!
```

Output-ul este util pentru că arată clar evoluția duelului și permite verificarea manuală a regulilor.

7 Concluzie

Proiectul implementează o simulare simplă de duel turn-based între două personaje. Codul respectă cerințele principale, include abilitățile speciale și are câteva extensii utile, precum simularea a 1000 de jocuri și alegerea modului de atribuire a abilităților.

Am încercat să păstrez codul cât mai clar, astfel încât să fie ușor de explicat și de modificat dacă apar cerințe noi.